# GMIU — DEPARTMENT OF COMPUTER ENGINEERING

## Academic Course: Cloud Computing (7th Semester)

**Practical Assignment Report: Multi-Cloud File Storage and Management System**

---

**Project Title:** Cloud File Manager: Architecture, Implementation, and Deployment on Render PaaS and Supabase Object Storage
**Student Name:** Chetan Bavaliya
**College/University:** GMIU
**Academic Year:** 2026
**Subject Code:** CC-701 (Cloud Computing)
**Submission Date:** October 28, 2026

---

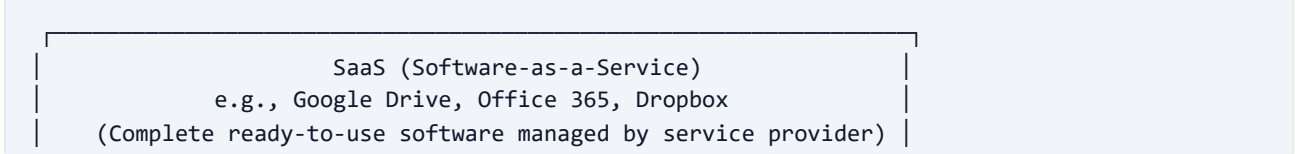
## 1. Objective

The primary objective of this project is to design, develop, test, and deploy a cloud-native web application demonstrating core cloud computing concepts without incurring financial costs:
1. **Compute Service**: Hosting an application backend using a modern Platform-as-a-Service (PaaS) provider.
2. **Storage Service**: Integrating an external managed cloud object storage service for binary assets rather than using local ephemeral server storage.
3. **Public Cloud Accessibility**: Serving application traffic securely over public Internet with automated HTTPS/TLS.
4. **Security & Access Control**: Implementing role-based credentials, private bucket access controls, and ephemeral cryptographic signed URLs.
5. **Resource Management & Monitoring**: Implementing strict request size boundaries (10 MB), auto-sleep management, and cloud infrastructure telemetry.

---

## 2. Introduction to Cloud Computing & Service Models

Cloud computing provides on-demand availability of computer system resources, especially data storage and computing power, without direct active management by the user. Cloud services are broadly categorized into three distinct abstraction models:



```
┌──────────────────────────────────────────────────────────────┐
│                     PaaS (Platform-as-a-Service)             │
│                 e.g., Render, Heroku, AWS Elastic Beanstalk  │
│       (Managed runtime, OS, web server, and build pipeline)  │
├──────────────────────────────────────────────────────────────┤
│                     IaaS (Infrastructure-as-a-Service)       │
│                   e.g., AWS EC2, GCP Compute Engine          │
│       (Virtual machines, raw networking, storage, firewalls) │
└──────────────────────────────────────────────────────────────┘
```

### Where This Project Fits in Cloud Computing Models

This project utilizes a **hybrid PaaS + BaaS (Backend-as-a-Service) architecture**:
- **Render Web Service (PaaS)**: Rather than provisioning raw Linux virtual machines (IaaS), Render abstracts the underlying operating system, kernel patching, firewall rules, and containerization. Developers push source code, and the platform handles compilation, container builds, and SSL termination.
- **Supabase Storage (BaaS / Storage-as-a-Service)**: Rather than attaching persistent block storage volumes (e.g., AWS EBS) or managing complex distributed file systems (e.g., NFS), the application delegates binary object lifecycle management to Supabase Storage (an S3-compatible managed object store with built-in CDN and cryptographic signing).

---

## 3. System Architecture

The application design follows the **Twelve-Factor App** methodology, strictly separating stateless compute from stateful persistence.

```
flowchart TD
    subgraph Client Tier
        Browser["🌐 Web Browser (Client)"]
    end

    subgraph "Render Cloud Platform (PaaS Compute - Singapore)"
        EdgeRouter["⚡ Cloud Edge Router (TLS / Let's Encrypt)"]
        GunicornWSGI["🦄 Gunicorn WSGI Server (2 Workers)"]
        FlaskEngine["🐍 Python 3.12 Flask Application"]
        EnvSecrets["🔐 Environment Secrets (SUPABASE_KEY, etc.)"]

        EdgeRouter -->|Reverse Proxy| GunicornWSGI
        GunicornWSGI --> FlaskEngine
        EnvSecrets -.->|Runtime Injection| FlaskEngine
    end

    subgraph "Supabase Cloud Platform (BaaS Object Storage)"
        StorageGateway["🚪 Supabase Storage REST Gateway"]
        AuthModule["🛡️ Service-Role Key Verification"]
        PrivateBucket[("🗄️ Private S3-Compatible Bucket ('uploads')")]

        StorageGateway --> AuthModule
```

```
        AuthModule --> PrivateBucket
    end

    Browser -->|1. HTTPS GET / POST| EdgeRouter
    FlaskEngine -->|2. Authenticated REST Call| StorageGateway
    StorageGateway -->|3. Signed Download URL (TTL: 3600s)| FlaskEngine
    FlaskEngine -->|4. Rendered HTML + Actions| Browser
    Browser -->|5. Direct Download Stream via Signed URL| StorageGateway
```

### Key Architectural Characteristics

1. **Stateless Compute**: Flask server instances hold zero permanent state on the local file system. If the Render instance is restarted, redeployed, or scaled, no stored user files are lost.
2. **Direct Storage Offload**: When users request a download, the application generates a temporary cryptographically signed URL. The actual binary payload streams directly from Supabase's high-throughput storage edge to the client, conserving Render's web server bandwidth and CPU cycles.

---

## 4. Tools and Services Used

| Category | Technology / Service | Role & Justification |
|---|---|---|
| **Compute / PaaS** | **Render.com** | Provides free Linux-based application hosting with automatic git deployment, zero-config SSL, and custom subdomain. |
| **Storage / BaaS** | **Supabase** | Open-source Firebase alternative providing managed S3-compatible private object storage with high reliability and zero cost for 1 GB. |
| **Backend Framework** | **Flask 3.x (Python 3.12)** | Lightweight, flexible micro-framework ideal for REST endpoints, file streaming, and templating. |
| **WSGI Server** | **Gunicorn 26.x** | Production-grade WSGI HTTP Server providing concurrency and process management for Python web applications. |
| **SDK / Client** | **supabase-py 2.x** | Official Python client library for communicating with Supabase Storage APIs. |
| **Version Control & CI/CD** | **GitHub** | Hosted Git repository integrated with Render for automated build and deployment webhooks. |
| **Testing** | **Pytest** | Automated unit testing framework using test doubles (mocks) for Supabase API interactions. |

---

## 5. Step-by-Step Implementation

### Step 1: Backend Architecture ( `app.py` )

- Configured Flask app with `MAX_CONTENT_LENGTH = 10 * 1024 * 1024` to cap uploads at 10 MB.
- Built helper functions for secure Supabase client initialization using `SUPABASE_URL` and `SUPABASE_KEY` .
- Implemented `/health` route returning JSON status `{"status": "ok"}` for cloud platform health probes.
- Created `/` route to retrieve the file catalog from the private bucket, calculate human-readable sizes (KB), filter hidden files ( `.emptyFolderPlaceholder` ), and generate signed download URLs valid for 3600 seconds.
- Created `/upload` route with Werkzeug `secure_filename()` sanitization, MIME-type resolution, and `upsert=true` bucket writing.
- Created `/delete/<path:name>` route with URL decoding and remote file deletion.

### Step 2: Front-End UI ( `templates/index.html` )

- Built a single-page responsive user interface with inline CSS featuring emerald green accents ( `#059669` ).
- Added real-time drag-and-drop file upload zone with file size warning feedback.
- Designed dynamic status banner: `Compute: Render Web Service | Storage: Supabase Storage | Bucket: uploads | Files: X` .
- Implemented action triggers for signed downloads and delete operations with confirmation modals.

### Step 3: Infrastructure-as-Code ( `render.yaml` )

- Declared infrastructure blueprint specifying Python runtime, build command `pip install -r requirements.txt` , start command `gunicorn app:app` , and environment variables with secret syncing disabled ( `sync: false` ).

### Step 4: Verification & Automated Testing ( `tests/test_app.py` )

- Created 9 comprehensive automated unit tests covering all routes, mocking remote Supabase storage responses, testing signed URL resolution for both `signedURL` and `signedUrl` response keys, and validating 413 file size exception handling.

---

## 6. Security Considerations

1. **Private Bucket Policy**:
   - The Supabase storage bucket `uploads` is marked strictly private. Direct public access attempts without a signed token are rejected with `403 Unauthorized` .

2. **Secrets Isolation**:
   - API keys and database credentials are never stored in source code. All secrets are read dynamically from runtime environment variables.
3. **Signed URLs**:
   - Downloads utilize time-limited HMAC-SHA256 signed URLs that expire after 3600 seconds (1 hour). After expiration, unauthorized parties cannot access the file link.
4. **Input Sanitization**:
   - All uploaded file paths are sanitized via `secure_filename()` to prevent directory traversal exploits ( `../../etc/shadow` ).
5. **Transport Layer Security (TLS)**:
   - Full end-to-end HTTPS encryption powered by Render's automated Let's Encrypt certificates.

---

## 7. Resource Management & Monitoring

1. **Upload Size Boundaries**:
   - Enforced 10 MB ceiling at the web server gateway. Oversized payloads are terminated early to preserve server memory and outbound network bandwidth.
2. **Ephemeral Disk Protection**:
   - Local disk writes are strictly prohibited; all file content is streamed directly to cloud storage, keeping the web container completely stateless.
3. **PaaS Free-Tier Life Cycle Management**:
   - Render automatically puts inactive free instances into sleep mode after 15 minutes of idle time. The application is architected to wake up cleanly on new HTTP requests.
4. **Cloud Telemetry**:
   - CPU and RAM utilization are tracked in the Render **Metrics** dashboard.
   - Storage volume and bandwidth are monitored via the Supabase **Usage & Storage** dashboard.

---

## 8. Challenges Faced & Solutions

| Challenge | Root Cause | Solution Implemented |
|---|---|---|
| **Ephemeral Container Storage** | Render containers wipe all local files on redeploy or container restart. | Offloaded 100% of binary file persistence to Supabase Storage bucket. |
| **Supabase SDK Key Discrepancies** | Different versions of `supabase-py` / `storage3` return signed URLs under either `signedURL` or `signedUrl` . | Implemented dual-key fallback checking in `extract_signed_url()` . |

| Challenge | Root Cause | Solution Implemented |
|---|---|---|
| **Large File Server Crashes** | Unrestricted file uploads can consume container RAM and cause out-of-memory kills. | Implemented Flask request limit `MAX_CONTENT_LENGTH` with custom 413 error handler. |
| **Hidden Bucket Metadata Objects** | Supabase occasionally creates `.emptyFolderPlaceholder` objects. | Added string filtering `not name.startswith(".")` in catalog listing loop. |

---

## 9. Conclusion

The **Cloud File Manager** project successfully demonstrates the deployment and operation of a multi-cloud distributed system combining compute (Render PaaS) and object storage (Supabase BaaS). By adhering to cloud engineering best practices—statelessness, time-limited cryptographic authorization, environment-based configuration, and automated continuous deployment—the system achieves high security, durability, and cost-efficiency ($0.00/month).

---

## 10. Screenshots of Deployed System

*(Screenshots will be attached upon deployment according to `docs/SCREENSHOT_CHECKLIST.md`)*

### Screenshot 1: Running Web Application

> **Description:** Shows the web browser displaying the live Cloud File Manager application on Render with the public URL bar visible, header tagline showing compute/storage metadata, uploaded files list, and download/delete action buttons.

```
[ INSERT SCREENSHOT 1 HERE: Live Application in Browser ]
```

### Screenshot 2: Supabase Storage Dashboard

> **Description:** Shows the Supabase Cloud dashboard displaying the private `uploads` storage bucket containing the uploaded files with matching file sizes and timestamps.

```
[ INSERT SCREENSHOT 2 HERE: Supabase Private Bucket View ]
```

### Screenshot 3: Render Web Service Dashboard

> **Description:** Shows the Render dashboard displaying the `cloud-file-manager` Web Service with `Live` status, `Singapore` region, `Free` instance tier, and recent commit hash.

```
[ INSERT SCREENSHOT 3 HERE: Render Service Live Dashboard ]
```

### Screenshot 4: Render Metrics / Logs Telemetry

> **Description:** Shows the Render `Logs` and `Metrics` tab confirming successful HTTP requests (200 OK, 302 redirects) and healthy memory/CPU consumption.

```
[ INSERT SCREENSHOT 4 HERE: Render Logs / Telemetry ]
```

### Screenshot 5: GitHub Repository & CI/CD

> **Description:** Shows the public GitHub repository `cloud-file-manager` containing the complete codebase, commit history, and Render deployment webhook badge.

```
[ INSERT SCREENSHOT 5 HERE: GitHub Repository Page ]
```